অধ্যায় ২৮

২৮

Professional JavaScript Deployment, Docker, CI/CD, Cloud Infrastructure এবং Production Monitoring

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Development ও Production পরিবেশের পার্থক্য এবং Twelve-Factor App-এর মূল নীতিগুলো ব্যাখ্যা করতে।
- Environment থেকে Configuration পড়ে শুরুতেই যাচাই করতে (Fail Fast), এবং গোপন মান Log থেকে লুকাতে।
- পুনরুৎপাদনযোগ্য Build Artifact, Lockfile, Commit SHA ও Image Digest-এর ভূমিকা বুঝতে।
- একটি নিরাপদ Dockerfile লিখতে: নির্দিষ্ট Base Image, Non-root ব্যবহারকারী, `.dockerignore`, HEALTHCHECK এবং Exec Form CMD।
- Docker Compose দিয়ে Production-এর মতো Local পরিবেশ সাজাতে।
- একটি CI Workflow লিখতে এবং Docker ছাড়াই Deploy File যাচাই করতে।
- Blue/Green, Rolling ও Canary Rollout তুলনা করতে, এবং সংখ্যা দিয়ে Canary-র সিদ্ধান্ত নিতে।
- Liveness ও Readiness আলাদা করতে এবং Graceful Shutdown বাস্তবায়ন করতে।
- Structured Log, Prometheus Format-এ Metrics ও Trace দিয়ে Application পর্যবেক্ষণ করতে।
- SLI, SLO, Error Budget ও Burn Rate হিসাব করে কাজে লাগানো Alert নকশা করতে।
- Secret, Supply Chain, Backup, RPO/RTO ও Incident Response-এর মূল অভ্যাস প্রয়োগ করতে।
- **DeployReady** নামে একটি Production-উপযোগী, পরীক্ষিত API সাজাতে।

পূর্বশর্ত

- অধ্যায় ২০: Secret, Token ও নিরাপদ Error বার্তা।
- অধ্যায় ২২: `node:test`, Integration Test।
- অধ্যায় ২৫: Node.js HTTP Server, Middleware, JSON Error চুক্তি, Graceful Shutdown-এর প্রথম পরিচয়।
- অধ্যায় ২৬: Migration ও Backup।
- অধ্যায় ২৭: দীর্ঘস্থায়ী সংযোগ ও Metrics-এর প্রয়োজন।
- Terminal-এ Command চালানোর অভ্যাস। Docker জানা আবশ্যক নয়।

এই অধ্যায় কেন গুরুত্বপূর্ণ

আপনার Computer-এ `npm start` চালালে Server চালু হয়—কাজ শেষ মনে হয়। কিন্তু একটি ভ্রমণ সংস্থার Operations দল যখন সকাল আটটায় হোটেল Booking যাচাইয়ের কাজ খোলেন, তখন প্রশ্নগুলো অন্য: নতুন সংস্করণ Deploy করতে গিয়ে কি চলমান অনুরোধ কেটে যাবে? কোন সংস্করণ এখন চলছে? ভুল Configuration নিয়ে Server চালু হয়ে চুপচাপ ভুল কাজ করছে না তো? রাত দুটোয় Error বাড়লে কে জানবে, আর কীভাবে বুঝবে কোন অনুরোধে সমস্যা? Database নষ্ট হলে কতটা তথ্য হারাবে, আর কতক্ষণে সেবা ফিরবে?

Deployment শুধু Code Server-এ তোলা নয়; এটি একটি পুনরাবৃত্তিযোগ্য, যাচাইযোগ্য এবং ফেরানো-যোগ্য প্রক্রিয়া। এই অধ্যায় সেই প্রক্রিয়ার প্রতিটি অংশ আসল Code দিয়ে দেখায়—Configuration থেকে Container, CI থেকে Canary, Health Check থেকে Error Budget—এবং শেষে একটি API তৈরি করে যা Production-এর এই প্রশ্নগুলোর উত্তর দিতে প্রস্তুত।

২৮.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
`process.env` পড়া (অধ্যায় ২৫)	পুরো Configuration শুরুতেই যাচাই, সব ভুল একসঙ্গে
Graceful Shutdown-এর পরিচয় (অধ্যায় ২৫)	Drain, Readiness 503, Timeout ও Container-এর Grace Period
Test (অধ্যায় ২২)	CI Pipeline, Smoke Test, Canary বিশ্লেষণ
Error-এর নিরাপদ বার্তা (অধ্যায় ২০)	Structured Log, Request ID, গোপন মান লুকানো
Metrics-এর প্রয়োজন (অধ্যায় ২৭)	Prometheus Format, Histogram, SLO, Burn Rate
Backup (অধ্যায় ২৬)	RPO, RTO, Restore Drill, Incident Response

> **নোট:** QA-যন্ত্রে Docker CLI ছিল, কিন্তু Docker Daemon চালু ছিল না; তাই `docker build` ও `docker compose up` চালানো হয়নি। পরিবর্তে Dockerfile একটি স্থির যাচাই Script দিয়ে, আর `compose.yaml` ও CI Workflow দুটি আলাদা YAML Parser দিয়ে যাচাই করা হয়েছে। CI Workflow GitHub-এ চালানো হয়নি—শুধু তার গঠন ও তার ডাকা প্রতিটি Script-এর অস্তিত্ব পরীক্ষিত। Application নিজে, তার Health Check, Shutdown ও Smoke Test আসলেই চালানো ও পরীক্ষিত।

২৮.২ Localhost থেকে Production: কী বদলায়

বিষয়	Local Development	Production-এর প্রত্যাশা
Configuration	`.env` বা হাতে লেখা মান	Platform-এর Configuration ও Secret Store, শুরুতেই যাচাই
Process	একটি Terminal	Supervisor বা Container Platform, স্বয়ংক্রিয় Restart
Data	Memory বা Local File	টেকসই Storage, Backup, Restore-এর অনুশীলন
Network	`localhost`	DNS, TLS, Load Balancer, Firewall
ব্যর্থতা	Developer নিজে দেখেন	Log, Metrics, Alert, Incident প্রক্রিয়া
Release	হাতে Restart	CI, ধাপে ধাপে Rollout, Rollback
সংস্করণ	"আমার Computer-এর Code"	নির্দিষ্ট Commit SHA ও Image Digest

এই সারণির প্রতিটি সারি একটি প্রশ্ন: "এটি ব্যর্থ হলে কীভাবে জানব, আর কীভাবে ফিরে যাব?" বাকি অধ্যায় সেই প্রশ্নের উত্তর।

২৮.৩ Twelve-Factor ও Configuration

Twelve-Factor App একটি প্রচলিত নীতিমালা, যার কয়েকটি এই অধ্যায়ের কেন্দ্রে: Configuration Code-এর বাইরে, Environment-এ (III); Process Stateless, অবস্থা থাকে Database-এ (VI); দ্রুত চালু ও সুশৃঙ্খলভাবে বন্ধ হওয়া (IX); Development ও Production যতটা সম্ভব একই (X); এবং Log একটি Event Stream, যা Process শুধু Standard Output-এ লেখে (XI)।

Configuration পড়ার সবচেয়ে বিপজ্জনক ভুল: `Number(process.env.PORT)` লিখে এগিয়ে যাওয়া। `PORT=abc` হলে মান `NaN`, আর Server হয়তো একটি এলোমেলো Port-এ চালু হয়। তার চেয়ে ভালো—শুরুতেই সব যাচাই, এবং ভুল থাকলে চালুই না হওয়া (**Fail Fast**):

config.mjs · JavaScript (অংশ ১ / ২ (Part 1 of 2)) চলবে: Node.js

```
// Twelve-Factor: সব Configuration Environment থেকে; শুরুতেই যাচাই।
// ভুল থাকলে সব ভুল একসঙ্গে জানিয়ে Process চালুই হবে না (Fail Fast)।
const LEVELS = ['debug', 'info', 'warn', 'error'];

function integer(env, name, fallback, min, max, errors) {
  const raw = env[name] ?? String(fallback);
  const value = Number(raw);
  if (!/^\d+$/.test(raw) || value < min || value > max) {
    errors.push(`${name} হতে হবে ${min} থেকে ${max}-এর পূর্ণসংখ্যা`);
  }
  return value;
}

// ...
```

config.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

export function loadConfig(env = process.env) {
  const errors = [];
  const nodeEnv = env.NODE_ENV ?? 'development';
  const config = {
    nodeEnv,
    port: integer(env, 'PORT', 3000, 0, 65535, errors),
    logLevel: env.LOG_LEVEL ?? 'info',
    shutdownTimeoutMs: integer(env, 'SHUTDOWN_TIMEOUT_MS', 10_000,
      1, 120_000, errors),
    drainDelayMs: integer(env, 'DRAIN_DELAY_MS', 0, 0, 60_000, errors),
    version: env.APP_VERSION ?? '0.0.0-dev',
    commit: env.GIT_SHA ?? 'unknown',
    opsToken: env.OPS_TOKEN,
  };
  if (!['development', 'test', 'production'].includes(nodeEnv)) {
    errors.push('NODE_ENV হতে হবে development, test বা production');
  }
  if (!LEVELS.includes(config.logLevel)) {
    errors.push(`LOG_LEVEL হতে হবে ${LEVELS.join(', ')}`);
  }
  if (nodeEnv === 'production' && (config.opsToken ?? '').length < 24) {
    errors.push('Production-এ OPS_TOKEN কমপক্ষে ২৪ অক্ষরের হতে হবে');
  }
  if (errors.length) {
    throw new Error(`Configuration ভুল:\n- ${errors.join('\n- ')}`);
  }
  return Object.freeze(config);
}
```

```
// Log বা Error-এ Config দেখাতে হলে গোপন মান লুকিয়ে।
export function describeConfig(config) {
  return { ...config, opsToken: config.opsToken ? '[REDACTED]' : undefined };
}
```

তিনটি সিদ্ধান্ত লক্ষ করুন। এক, ভুলগুলো একটি Array-তে জমে, তাই Operator একবারে সব ঠিক করতে পারেন—একটি ঠিক করে Deploy, তারপর পরেরটি, এমন নয়। দুই, Production-এ `OPS_TOKEN` বাধ্যতামূলক এবং দীর্ঘ; Development-এ ঐচ্ছিক। তিন, `describeConfig` গোপন মান লুকিয়ে দেয়, যাতে "চালু হলো, এই Configuration দিয়ে" Log-এ Token ফাঁস না হয়।

config-fail-fast.mjs · JavaScript — চলবে: Node.js

```
import { loadConfig, describeConfig }
  from '../project/deploy-ready/src/config.mjs';

const environments = {
  'Local, কিছুই দেওয়া নেই': {},
  'ভুল টাইপ': { PORT: 'abc', LOG_LEVEL: 'verbose' },
  'Production, Token নেই': { NODE_ENV: 'production', PORT: '8080' },
  'Production, ঠিকঠাক': { NODE_ENV: 'production', PORT: '8080',
    OPS_TOKEN: 'k'.repeat(32), GIT_SHA: '9f2c1ab' },
};

for (const [name, env] of Object.entries(environments)) {
  try {
    const config = describeConfig(loadConfig(env));
    console.log(`${name}: চালু`, { port: config.port,
      env: config.nodeEnv, token: config.opsToken });
  } catch (error) {
    console.log(`${name}: থামল\n  ${error.message.replace(/\n/g, '\n  ')}`);
  }
}
```

Output

```
Local, কিছুই দেওয়া নেই: চালু { port: 3000, env: 'development', token: undefined }
ভুল টাইপ: থামল
  Configuration ভুল:
  - PORT হতে হবে 0 থেকে 65535-এর পূর্ণসংখ্যা
  - LOG_LEVEL হতে হবে debug, info, warn, error
Production, Token নেই: থামল
  Configuration ভুল:
  - Production-এ OPS_TOKEN কমপক্ষে ২৪ অক্ষরের হতে হবে
Production, ঠিকঠাক: চালু { port: 8080, env: 'production', token: '[REDACTED]' }
```

`.env` File Git-এ না রাখা জরুরি, কিন্তু তা একা Secret Management নয়। Production-এ গোপন মান থাকে Platform-এর Secret Store-এ (Cloud Provider-এর Secret Manager, Kubernetes Secret, বা Vault), যেখানে কে পড়তে পারে তা নিয়ন্ত্রিত, পরিবর্তনের ইতিহাস থাকে, এবং মান নিয়মিত বদলানো (Rotation) যায়।

২৮.৪ Build Artifact, Lockfile ও পুনরুৎপাদনযোগ্যতা

একটি পরিচ্ছন্ন Release-প্রবাহ: Source Code, তারপর স্বয়ংক্রিয় যাচাই, তারপর একটি **অপরিবর্তনীয় Build Artifact** (যেমন Docker Image), এবং সেই একই Artifact প্রথমে Staging-এ, পরে Production-এ। Production-এর জন্য আলাদা করে আবার Build করবেন না—তাহলে যা পরীক্ষা হয়েছে আর যা চলছে, তা এক থাকে না।

- **Lockfile** (`package-lock.json`) প্রতিটি Dependency-র হুবহু সংস্করণ ও Checksum ধরে রাখে। CI ও Image-এ `npm ci` ব্যবহার করুন: এটি শুধু Lockfile মেনে Install করে, আর Lockfile ও `package.json` না মিললে থেমে যায়। `npm install` Lockfile হালনাগাদ করতে পারে।
- **Release-এর পরিচয়:** Git Commit SHA এবং Image Digest (`sha256:...`)। `latest` Tag একটি চলমান লক্ষ্য: আজ এক Image, কাল আরেক। Rollback-এর জন্য জানতে হবে আগে ঠিক কোনটি চলছিল।
- **একই Runtime:** Node.js-এর Major Version, Operating System ও Native Module—এগুলোও Artifact-এর অংশ। Container এর বড় সুবিধা।

DeployReady-এর কোনো Runtime Dependency নেই, তাই Lockfile নেই। বাস্তব Project-এ Lockfile Git-এ রাখুন এবং Dependency-র নিরাপত্তা Scan CI-তে চালান।

২৮.৫ Docker: Image, Container ও Dockerfile

Image হলো একটি File System-এর স্তরে স্তরে সাজানো ছবি, সঙ্গে চালানোর নির্দেশ—অপরিবর্তনীয়। **Container** হলো সেই Image থেকে চালু হওয়া একটি বিচ্ছিন্ন Process। এক Image থেকে অনেক Container চলতে পারে। **Dockerfile** Image তৈরির নির্দেশমালা; প্রতিটি নির্দেশ একটি স্তর (Layer), আর যে স্তর বদলায়নি, Docker তা আগের Build থেকে আবার ব্যবহার করে (Layer Cache)।

DeployReady-এর Dockerfile:

Terminal

```
cat Dockerfile
```

Output

```
# syntax=docker/dockerfile:1
# Production-এ Tag-এর বদলে Digest দিয়ে Pin করুন: node:22-alpine@sha256:<digest>
FROM node:22-alpine

ENV NODE_ENV=production
```

```
WORKDIR /app

# প্রথমে যা কম বদলায় (package.json), পরে Code: Layer Cache কাজে লাগে।
# এই Project-এ কোনো Runtime Dependency নেই। থাকলে এখানে:
#   COPY package-lock.json ./  এবং  RUN npm ci --omit=dev
COPY --chown=node:node package.json ./
COPY --chown=node:node main.mjs healthcheck.mjs ./
COPY --chown=node:node src ./src

# Root হিসেবে নয়: Image-এ আগে থেকে থাকা 'node' ব্যবহারকারী।
USER node
EXPOSE 3000

HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retries=3 \
  CMD ["node", "healthcheck.mjs"]

# Exec Form: node নিজেই PID 1, তাই SIGTERM সরাসরি পায়।
# "npm start" ব্যবহার করবেন না; npm Signal ঠিকমতো পৌঁছে দেয় না।
STOPSIGNAL SIGTERM
CMD ["node", "main.mjs"]
```

প্রতিটি সিদ্ধান্তের কারণ:

নির্দেশ	কেন
`FROM node:22-alpine`	ছোট Base Image, নির্দিষ্ট Major Version; Production-এ Digest দিয়ে Pin
`package.json` আগে, Code পরে	Code বদলালেও Dependency-র স্তর Cache থেকে আসে
নির্দিষ্ট File `COPY`	`COPY . .` অপ্রয়োজনীয় ও গোপন File-ও ঢুকিয়ে দিতে পারে
`USER node`	Container-এর ভেতরে কেউ ঢুকলেও Root ক্ষমতা পায় না
`HEALTHCHECK`	Docker নিজেই জানে Container সুস্থ কি না
Exec Form `CMD ["node", ...]`	`node` PID 1, তাই Platform-এর SIGTERM সরাসরি পায়

`npm start` দিয়ে চালালে PID 1 হয় `npm`, এবং `npm` SIGTERM Application-এ পৌঁছে দেয় না বা দেরিতে দেয়—ফলে Graceful Shutdown কখনো ঘটে না, Platform কিছুক্ষণ পরে জোর করে মেরে ফেলে। Alpine-এ Native Module-এর সামঞ্জস্য (musl libc) যাচাই করতে হয়; সমস্যা হলে `node:22-slim` (Debian-ভিত্তিক)।

`.dockerignore` Build-এর সময় কোন File Docker-এ পাঠানো হবে না তা বলে:

Terminal

```
cat .dockerignore
```

Output

```
node_modules
npm-debug.log
.git
.github
.env
.env.*
!.env.example
tests
coverage
*.md
compose.yaml
Dockerfile
```

`node_modules` বাদ যায় কারণ Host-এর (যেমন Windows-এর) Native Module Linux Container-এ কাজ নাও করতে পারে, আর Build Context অকারণে বড় হয়। `.env` ও `.git` বাদ যায় কারণ গোপন মান বা পুরো ইতিহাস Image-এ ঢুকে যেতে পারে—আর Image একবার Registry-তে গেলে তা ফেরানো কঠিন।

Docker-এর `HEALTHCHECK` যে Script চালায়, তা Image-এ আগে থেকে থাকা Node.js দিয়েই লেখা, যাতে `curl` বা `wget` আলাদা করে লাগে না:

healthcheck.mjs · JavaScript — চলবে: Node.js

```javascript
// Docker HEALTHCHECK: Container-এর ভেতর থেকে Liveness যাচাই।
// curl বা wget-এর ওপর নির্ভর নয়; Image-এ Node.js তো আছেই।
const port = process.env.PORT ?? '3000';
try {
  const res = await fetch(`http://127.0.0.1:${port}/health/live`,
    { signal: AbortSignal.timeout(2000) });
  process.exit(res.ok ? 0 : 1);
} catch {
  process.exit(1);
}
```

এটি সরাসরি চালানো হয়নি এই Listing থেকে, কারণ এটি একটি চলমান Server চায়; Project-এর Test এটি আসল Server-এর বিপরীতে চালিয়ে দেখে যে চালু Server-এ Exit Code 0 এবং বন্ধ Port-এ 1।

Security-র ভারসাম্য: ছোট Image আক্রমণের সুযোগ কমায়, কিন্তু Image Scanning, হালনাগাদ Base Image এবং Dependency পর্যালোচনার বিকল্প নয়।

২৮.৬ Docker Compose: Local-এ Production-এর অনুকরণ

Compose একটি YAML File-এ এক বা একাধিক Service-এর বর্ণনা দেয়—Build, Port, Environment, Restart নীতি, বন্ধের সময়সীমা। Development-এ ও ছোট Deployment-এ কার্যকর; একাধিক Server-এ ছড়ানো, Autoscaling ও উচ্চ Availability-র জন্য Kubernetes বা Managed Container Platform লাগে।

compose.yaml · YAML — চলবে: Node.js

```
services:
  api:
    build: .
    image: deploy-ready:local
    # শুধু নিজের Computer থেকে পৌঁছানো যাবে, Network-এর অন্যদের থেকে নয়।
    ports:
      - "127.0.0.1:3000:3000"
    # গোপন মান (OPS_TOKEN) .env-এ; .env কখনো Git-এ নয় (.env.example দেখুন)।
    env_file: .env
    environment:
      NODE_ENV: production
      PORT: "3000"
      DRAIN_DELAY_MS: "5000"
      SHUTDOWN_TIMEOUT_MS: "15000"
    # Drain (৫ সে.) + Shutdown Timeout (১৫ সে.) < Grace Period (৩০ সে.)
    stop_grace_period: 30s
    restart: unless-stopped
    read_only: true
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
```

তিনটি জিনিস বিশেষভাবে দেখুন। **Port** শুধু `127.0.0.1`-এ: `"3000:3000"` লিখলে Docker সব Network Interface-এ খোলে, আর Docker নিজের Firewall নিয়ম যোগ করে বলে Host-এর Firewall অনেক সময় তা আটকায় না। **Secret** আসে `env_file` থেকে, Compose File-এ নয়। আর **সময়ের হিসাব**: Drain (৫ সেকেন্ড) + Shutdown Timeout (১৫ সেকেন্ড) = ২০ সেকেন্ড, যা `stop_grace_period` (৩০ সেকেন্ড)-এর কম। উল্টো হলে Docker Application-এর Shutdown শেষ হওয়ার আগেই SIGKILL পাঠাবে। `read_only`, `cap_drop` ও `no-new-privileges` Container-এর ভেতরের ক্ষমতা আরও কমায়।

২৮.৭ CI: প্রতিটি পরিবর্তন যাচাই

Continuous Integration মানে প্রতিটি Push ও Pull Request-এ স্বয়ংক্রিয়ভাবে Test, Syntax ও নিরাপত্তা যাচাই, এবং Image Build। কোনো ধাপ ব্যর্থ হলে Merge বা Release আটকে যায়। GitHub Actions-এর Workflow:

ci.yml · YAML অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

# সবচেয়ে কম অনুমতি: Code পড়া ছাড়া কিছু নয়।
permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      # Production Pipeline-এ Tag-এর বদলে পূর্ণ Commit SHA দিয়ে Pin করুন।
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: npm test
      - run: npm run check

# ...
```

দ্বিতীয় Job Image Build করে, চালায় এবং Smoke Test দিয়ে যাচাই করে। Token প্রতিবার নতুন করে তৈরি হয়, কোথাও সংরক্ষিত নয়:

ci.yml · YAML অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```yaml
# ...

  image:
    needs: test
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - name: Build image (tag = commit SHA)
```

```
        run: docker build -t deploy-ready:${{ github.sha }} .
      - name: Run image and smoke test
        run: |
          export OPS_TOKEN="$(openssl rand -hex 16)"
          docker run -d --name api -p 3000:3000 \
            -e OPS_TOKEN -e GIT_SHA=${{ github.sha }} \
            deploy-ready:${{ github.sha }}
          for i in $(seq 1 20); do
            curl -fs http://127.0.0.1:3000/health/ready && break
            sleep 1
          done
          node scripts/smoke.mjs http://127.0.0.1:3000
      - name: Logs on failure
        if: failure()
        run: docker logs api
```

নিরাপত্তার অভ্যাসগুলো: `permissions: contents: read` —Workflow-এর Token শুধু Code পড়তে পারে; `timeout-minutes` —আটকে থাকা Job চিরকাল চলে না; `concurrency` —একই Branch-এ নতুন Push এলে পুরোনো চালনা বাতিল; `needs: test` —Test পাস না করলে Image-এর কাজ শুরুই হয় না। Third-party Action (`actions/checkout@v4`) Tag দিয়ে উল্লেখ করা—Tag সরানো যায়, তাই Production Pipeline-এ পূর্ণ Commit SHA দিয়ে Pin করুন। মনে রাখবেন, CI সবুজ মানে Production-এর জন্য প্রস্তুত নয়; এটি শুধু বলে যে আপনার জানা যাচাইগুলো পার হয়েছে।

Docker ছাড়াও Deploy File-এর সাধারণ ভুল ধরা যায়। `npm run check` এই Script চালায়:

check-deploy.mjs · JavaScript · অংশ ১ / ২ (Part 1 of 2) · চলবে: Node.js

```
// Docker ছাড়াই Deploy File-এর সাধারণ ভুল ধরা (CI-তে Build-এর আগে)।
import { readFileSync } from 'node:fs';

const read = (file) => readFileSync(new URL(`../${file}`, import.meta.url),
  'utf8');
const problems = [];
const rule = (ok, message) => { if (!ok) problems.push(message); };

// Dockerfile: '\' দিয়ে ভাঙা Line জোড়া লাগিয়ে প্রতিটি Instruction আলাদা।
const steps = read('Dockerfile').replace(/\\\n/g, ' ').split('\n')
  .map((line) => line.trim()).filter((line) => line && !line.startsWith('#'))
  .map((line) => {
    const [op, ...rest] = line.split(/\s+/);
    return { op: op.toUpperCase(), args: rest.join(' ') };
  });
const find = (op) => steps.filter((s) => s.op === op);
const index = (op) => steps.findIndex((s) => s.op === op);

rule(find('FROM').length > 0 && !/:latest\b/.test(find('FROM')[0].args)
  && /:/.test(find('FROM')[0].args), 'FROM-এ নির্দিষ্ট Tag বা Digest');
```

```
rule(find('USER').some((s) => !/^(root|0)$/.test(s.args)),
  'USER: root ছাড়া অন্য ব্যবহারকারী');
rule(index('USER') !== -1 && index('USER') < index('CMD'),
  'USER থাকতে হবে CMD-এর আগে');
rule(find('CMD').at(-1)?.args.startsWith('['), 'CMD Exec Form-এ (JSON Array)');
rule(!/npm/.test(find('CMD').at(-1)?.args ?? ''), 'CMD-এ npm নয়, সরাসরি node');
rule(find('HEALTHCHECK').length === 1, 'একটি HEALTHCHECK');
rule(!steps.some((s) => ['ENV', 'ARG'].includes(s.op)
  && /(SECRET|TOKEN|PASSWORD|KEY)\s*=/i.test(s.args)),
  'ENV বা ARG-এ কোনো গোপন মান নয়');
rule(!find('COPY').some((s) => /^\.\s+\.$/.test(s.args.replace(/--\S+\s*/g, ''))),
  'COPY . . নয়; শুধু প্রয়োজনীয় File');

// ...
```

check-deploy.mjs · JavaScript (অংশ ২ / ২ (Part 2 of 2)) চলবে: Node.js

```
// ...

const ignore = read('.dockerignore').split('\n').map((l) => l.trim());
for (const entry of ['node_modules', '.git', '.env']) {
  rule(ignore.includes(entry), `.dockerignore-এ ${entry}`);
}

const compose = read('compose.yaml');
rule(/"127\.0\.0\.1:\d+:\d+"/.test(compose), 'Compose Port শুধু 127.0.0.1-এ');
rule(/stop_grace_period:/.test(compose), 'Compose-এ stop_grace_period');
rule(!/OPS_TOKEN:/.test(compose), 'Compose File-এ Token-এর মান নয়');

for (const p of problems) console.log(`FAIL ${p}`);
console.log(`Instruction: ${steps.length}, সমস্যা: ${problems.length}`);
process.exit(problems.length ? 1 : 0);
```

Output

```
Instruction: 11, সমস্যা: 0
```

এটি একটি পূর্ণ Linter নয় (যেমন `hadolint`), কিন্তু এই Project-এর নিয়মগুলো—Non-root, Exec Form, HEALTHCHECK, গোপন মান নয়, `.dockerignore`—প্রতিটি পরিবর্তনে স্বয়ংক্রিয়ভাবে রক্ষা করে।

২৮.৮ CD: Staging, Rollout ও Rollback

Continuous Delivery-তে প্রতিটি সফল Pipeline একটি Release-যোগ্য Artifact তৈরি করে, আর Production-এ যাওয়ার সিদ্ধান্ত মানুষের (একটি Approval)। **Continuous Deployment**-এ সেই সিদ্ধান্তও স্বয়ংক্রিয়। একটি সাধারণ প্রবাহ:

Folder-এর কাঠামো

```
Pull Request -> Test -> Image Build -> Security Scan
  -> Staging Deploy -> Smoke Test -> Approval
  -> Production Rollout (ধাপে ধাপে) -> পর্যবেক্ষণ -> প্রয়োজনে Rollback
```

কৌশল	কীভাবে	সুবিধা	মূল্য
Rolling	Instance-গুলো একে একে নতুন সংস্করণে	অতিরিক্ত Server লাগে না	কিছুক্ষণ দুই সংস্করণ একসঙ্গে; Rollback ধীর
Blue/Green	পুরো নতুন পরিবেশ, তারপর Traffic এক ধাক্কায় বদল	তাৎক্ষণিক Rollback	দ্বিগুণ Infrastructure
Canary	প্রথমে অল্প Traffic (যেমন ৫%) নতুন সংস্করণে, মেপে বাড়ানো	ক্ষতি সীমিত, আসল Traffic-এ যাচাই	Metrics ও স্বয়ংক্রিয় বিশ্লেষণ লাগে

Canary-র সিদ্ধান্ত অনুভূতিতে নয়, সংখ্যায়:

canary-decision.mjs · JavaScript — চলবে: Node.js

```javascript
// Canary: নতুন সংস্করণে অল্প Traffic; পুরোনোর (Baseline) সঙ্গে তুলনা করে
// এগোনো, অপেক্ষা বা ফিরে যাওয়া। সিদ্ধান্ত সংখ্যা দিয়ে, অনুমানে নয়।
function decide(baseline, canary, { minRequests = 500, maxErrorRatio = 1.5,
  maxLatencyMs = 50 } = {}) {
  if (canary.requests < minRequests) return 'wait: যথেষ্ট Traffic নেই';
  const errorRate = (w) => w.errors / w.requests;
  const ratio = errorRate(canary) / Math.max(errorRate(baseline), 0.0001);
  if (ratio > maxErrorRatio) {
    return `rollback: Error ${ratio.toFixed(1)} গুণ`;
  }
  if (canary.p95Ms - baseline.p95Ms > maxLatencyMs) {
    return `rollback: p95 ${canary.p95Ms - baseline.p95Ms} ms বেশি`;
  }
  return 'promote: পরের ধাপে (যেমন ৫% থেকে ২৫%)';
}

const baseline = { requests: 95_000, errors: 95, p95Ms: 180 };
const cases = {
  'প্রথম ৫ মিনিট': { requests: 120, errors: 0, p95Ms: 170 },
  'সুস্থ': { requests: 5_000, errors: 6, p95Ms: 190 },
  'Error বেশি': { requests: 5_000, errors: 40, p95Ms: 185 },
  'ধীর': { requests: 5_000, errors: 4, p95Ms: 320 },
};
for (const [name, canary] of Object.entries(cases)) {
  console.log(`${name}: ${decide(baseline, canary)}`);
}
```

Output

```
প্রথম ৫ মিনিট: wait: যথেষ্ট Traffic নেই
সুস্থ: promote: পরের ধাপে (যেমন ৫% থেকে ২৫%)
Error বেশি: rollback: Error 8.0 গুণ
ধীর: rollback: p95 140 ms বেশি
```

প্রথম ক্ষেত্রে ১২০টি অনুরোধে শূন্য Error কিছুই প্রমাণ করে না—তাই অপেক্ষা। "সুস্থ" Canary-র Error Rate ০.১২%, Baseline-এর ০.১%-এর কাছাকাছি, তাই এগোনো। শেষ দুটিতে হয় Error, নয় দেরি স্পষ্টভাবে খারাপ—ফিরে যাওয়া।

Database Migration-এর সতর্কতা: Application Rollback সহজ, কিন্তু Database-এর পরিবর্তন সহজে ফেরে না। Rollout-এর সময় পুরোনো ও নতুন দুই সংস্করণ একই Database ব্যবহার করে, তাই প্রতিটি Migration পশ্চাৎ-সামঞ্জস্যপূর্ণ হতে হবে। নিয়ম **Expand/Contract**: প্রথমে যোগ (নতুন Column, ঐচ্ছিক), তারপর Code নতুনটি ব্যবহার করে, তারপর পুরোনো Data স্থানান্তর, এবং অনেক পরে—সব পুরোনো সংস্করণ চলে গেলে—পুরোনো Column মোছা।

২৮.৯ Liveness, Readiness ও Graceful Shutdown

দুটি Health Endpoint দুটি ভিন্ন প্রশ্নের উত্তর দেয়:

- **Liveness** (`/health/live`): "Process কি জীবিত?" ব্যর্থ হলে Platform Container Restart করে। তাই এখানে Database যাচাই করবেন না—Database বন্ধ হলে সব Container একসঙ্গে Restart হয়ে অবস্থা আরও খারাপ হবে।
- **Readiness** (`/health/ready`): "এখন কি Traffic নিতে পারি?" ব্যর্থ হলে Load Balancer এই Instance-এ নতুন অনুরোধ পাঠানো বন্ধ করে, কিন্তু Restart করে না। এখানে প্রয়োজনীয় Dependency সময়সীমা দিয়ে যাচাই করা যায়, এবং Shutdown শুরু হলে এটি 503 দেয়।

Server-এর জীবনচক্র:

server.mjs · JavaScript — চলবে: Node.js

```
import http from 'node:http';
import { setTimeout as sleep } from 'node:timers/promises';
import { createApp } from './app.mjs';

// HTTP Server-এর জীবনচক্র: চালু করা এবং সুশৃঙ্খলভাবে বন্ধ করা।
export async function startServer({ config, logger, metrics, store }) {
  const app = createApp({ config, logger, metrics, store });
  const server = http.createServer(app.handle);
  server.keepAliveTimeout = 5_000;
  server.headersTimeout = 10_000;
  server.requestTimeout = 30_000;
  await new Promise((resolve) => server.listen(config.port, resolve));
  const { port } = server.address();
  logger.info('server.started', { port, version: config.version,
    commit: config.commit });
```

```
  let closing;
  // ১) Readiness 503, ২) Load Balancer-এর জন্য অপেক্ষা, ৩) নতুন সংযোগ বন্ধ,
  // ৪) চলমান অনুরোধ শেষ হতে দেওয়া, ৫) সময় পেরোলে জোর করে বন্ধ।
  function shutdown(reason) {
    closing ??= (async () => {
      app.state.draining = true;
      logger.info('shutdown.started', { reason,
        inFlight: app.state.inFlight });
      await sleep(config.drainDelayMs);
      let forced = false;
      const closed = new Promise((resolve) => server.close(resolve));
      server.closeIdleConnections();
      const timer = setTimeout(() => {
        forced = true;
        server.closeAllConnections();
      }, config.shutdownTimeoutMs);
      await closed;
      clearTimeout(timer);
      logger.info('shutdown.complete', { forced });
      return { forced };
    })();
    return closing;
  }

  return { server, port, app, shutdown };
}
```

Shutdown-এর পাঁচ ধাপের কারণ: Platform SIGTERM পাঠানোর মুহূর্তে Load Balancer হয়তো তখনো এই Instance-এ অনুরোধ পাঠাচ্ছে। তাই প্রথমে Readiness 503, তারপর `drainDelayMs` অপেক্ষা যাতে Load Balancer তা লক্ষ করে এই Instance-কে তালিকা থেকে সরায়। এরপর `server.close()` নতুন সংযোগ নেওয়া বন্ধ করে, আর অলস Keep-Alive সংযোগগুলো বন্ধ হয়। চলমান অনুরোধ শেষ হলে `close`-এর Callback আসে। কোনো অনুরোধ আটকে থাকলে Timeout-এর পর জোর করে সব বন্ধ—এবং `forced: true` জানায় যে কিছু অনুরোধ অসমাপ্ত রইল, তাই Exit Code 1। `closing ??=` নিশ্চিত করে যে দ্বিতীয় Signal (অধৈর্য Operator-এর দ্বিতীয় Ctrl+C) একই প্রক্রিয়া আবার শুরু করে না।

`keepAliveTimeout`, `headersTimeout` ও `requestTimeout` সীমা দেয় যাতে একটি ধীর বা দুষ্ট Client সংযোগ চিরকাল ধরে রাখতে না পারে। Load Balancer-এর পেছনে থাকলে `keepAliveTimeout` Load Balancer-এর Idle Timeout-এর চেয়ে বেশি রাখুন, নইলে মাঝে মাঝে "Connection reset" Error দেখা যায়।

২৮.১০ Logs, Metrics ও Traces

পর্যবেক্ষণের (**Observability**) তিনটি সংকেত: **Log**—নির্দিষ্ট ঘটনার বিবরণ; **Metrics**—সময়ের সঙ্গে সংখ্যার প্রবণতা; **Trace**—একটি অনুরোধ একাধিক Service-এর মধ্য দিয়ে কোন পথে গেল। Google SRE-এর চারটি **Golden Signal**: Latency, Traffic, Error, Saturation।

Structured Log: "User rafi logged in" ধরনের মুক্ত বাক্য মানুষ পড়তে পারে, কিন্তু Log-ব্যবস্থা খুঁজতে বা গুনতে পারে না। প্রতি Line-এ একটি JSON Object হলে "শেষ এক ঘণ্টায় কতগুলো `auth.failed`" বা "এই requestId-এর সব Line" এক Query-তে পাওয়া যায়:

logger.mjs · JavaScript — চলবে: Node.js

```
// Structured Log: প্রতি Line একটি JSON Object, যাতে Log-ব্যবস্থা খুঁজতে
// ও গুনতে পারে। গোপন Key-গুলোর মান কখনো লেখা হয় না।
const LEVELS = { debug: 10, info: 20, warn: 30, error: 40 };
const SECRET = /authorization|password|token|secret|cookie/i;

function redact(value) {
  if (Array.isArray(value)) return value.map(redact);
  if (value === null || typeof value !== 'object') return value;
  return Object.fromEntries(Object.entries(value).map(([key, inner]) =>
    [key, SECRET.test(key) ? '[REDACTED]' : redact(inner)]));
}

export function createLogger({ level = 'info', base = {},
  write = (line) => process.stdout.write(`${line}\n`),
  now = () => new Date() } = {}) {
  const min = LEVELS[level];
  const log = (lvl) => (event, fields = {}) => {
    if (LEVELS[lvl] < min) return;
    write(JSON.stringify({ time: now().toISOString(), level: lvl, event,
      ...redact(base), ...redact(fields) }));
  };
  return {
    debug: log('debug'), info: log('info'),
    warn: log('warn'), error: log('error'),
    child: (fields) => createLogger({ level, write, now,
      base: { ...base, ...fields } }),
  };
}
```

structured-log.mjs · JavaScript — চলবে: Node.js

```javascript
import { createLogger } from '../project/deploy-ready/src/logger.mjs';

// স্থির সময়, যাতে আউটপুট প্রতিবার এক।
let tick = 0;
const now = () => new Date(Date.UTC(2026, 9, 18, 10, 0, tick++));
// আসলে প্রতিটি Log একটি JSON Line; এখানে পড়ার সুবিধায় Object আকারে ছাপা।
const write = (line) => console.log(JSON.parse(line));
const logger = createLogger({ level: 'info', now, write,
  base: { service: 'deploy-ready' } });

const request = logger.child({ requestId: 'req-7f3a' });
request.debug('cache.miss', { key: 'jobs' }); // level info: লেখা হবে না
request.info('job.created', { jobId: 42, priority: 'high' });
request.warn('auth.failed', { authorization: 'Bearer s3cr3t', ip: '10.0.0.8' });
logger.error('db.timeout', { query: 'list-jobs', durationMs: 5003 });
```

Output

```
{
  time: '2026-10-18T10:00:00.000Z',
  level: 'info',
  event: 'job.created',
  service: 'deploy-ready',
  requestId: 'req-7f3a',
  jobId: 42,
  priority: 'high'
}
{
  time: '2026-10-18T10:00:01.000Z',
  level: 'warn',
  event: 'auth.failed',
  service: 'deploy-ready',
  requestId: 'req-7f3a',
  authorization: '[REDACTED]',
  ip: '10.0.0.8'
}
{
  time: '2026-10-18T10:00:02.000Z',
  level: 'error',
  event: 'db.timeout',
  service: 'deploy-ready',
  query: 'list-jobs',
  durationMs: 5003
}
```

`debug` Line লেখা হয়নি কারণ Level `info`। `child` Logger প্রতিটি Line-এ `requestId` বসায়, তাই একটি অনুরোধের সব ঘটনা একসঙ্গে খোঁজা যায়। `authorization`-এর মান কখনো লেখা হয় না। Log-এ Password, Token, পূর্ণ কার্ড নম্বর বা অপ্রয়োজনীয় ব্যক্তিগত তথ্য রাখবেন না—Log অনেক মানুষ ও অনেক ব্যবস্থার হাতে যায়, আর দীর্ঘ সময় থাকে।

Metrics: Prometheus একটি বহুল ব্যবহৃত Metrics ব্যবস্থা, যা নির্দিষ্ট বিরতিতে Application-এর `/metrics` Endpoint থেকে একটি সহজ Text Format পড়ে। **Counter** শুধু বাড়ে (মোট অনুরোধ); **Gauge** বাড়ে-কমে (চলমান অনুরোধ); **Histogram** মানগুলোকে Bucket-এ গোনে, যা থেকে p95-এর মতো Percentile হিসাব করা যায়:

metrics.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
// Prometheus Text Exposition Format-এ Metrics: Counter ও Histogram।
const BUCKETS = [0.005, 0.01, 0.05, 0.1, 0.5, 1, 5];
const HIST = 'http_request_duration_seconds';

const labelText = (labels) => Object.entries(labels)
  .map(([key, value]) => `${key}="${String(value).replace(/"/g, '\\"')}"`)
  .join(',');

export function createMetrics() {
  const requests = new Map();
  const durations = new Map();
  const gauges = new Map();

// ...
```

metrics.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  return {
    observeRequest({ method, route, status, seconds }) {
      const key = labelText({ method, route, status });
      requests.set(key, (requests.get(key) ?? 0) + 1);
      const hkey = labelText({ method, route });
      const h = durations.get(hkey)
        ?? { counts: BUCKETS.map(() => 0), sum: 0, count: 0 };
      BUCKETS.forEach((le, i) => { if (seconds <= le) h.counts[i] += 1; });
      h.sum += seconds;
      h.count += 1;
      durations.set(hkey, h);
    },
    setGauge(name, value) {
      gauges.set(name, value);
    },
    render() {
      const lines = ['# HELP http_requests_total HTTP অনুরোধের সংখ্যা',
```

```
        '# TYPE http_requests_total counter'];
      for (const [key, n] of requests) {
        lines.push(`http_requests_total{${key}} ${n}`);
      }
      lines.push(`# HELP ${HIST} অনুরোধের সময় (সেকেন্ড)`,
        `# TYPE ${HIST} histogram`);
      for (const [key, h] of durations) {
        BUCKETS.forEach((le, i) => lines.push(
          `${HIST}_bucket{${key},le="${le}"} ${h.counts[i]}`));
        lines.push(`${HIST}_bucket{${key},le="+Inf"} ${h.count}`,
          `${HIST}_sum{${key}} ${h.sum}`,
          `${HIST}_count{${key}} ${h.count}`);
      }
      for (const [name, value] of gauges) {
        lines.push(`# TYPE ${name} gauge`, `${name} ${value}`);
      }
      return `${lines.join('\n')}\n`;
    },
  };
}
```

metrics-format.mjs · JavaScript — চলবে: Node.js

```
import { createMetrics } from '../project/deploy-ready/src/metrics.mjs';

const metrics = createMetrics();
const seen = [
  ['GET', 200, 0.003], ['GET', 200, 0.04], ['GET', 200, 0.3],
  ['POST', 201, 0.02], ['POST', 422, 0.004], ['GET', 500, 1.7],
];
for (const [method, status, seconds] of seen) {
  metrics.observeRequest({ method, route: '/jobs', status, seconds });
}
// শুধু GET-এর Histogram ও সব Counter দেখানো (পুরো আউটপুট দীর্ঘ)।
const text = metrics.render().split('\n')
  .filter((line) => !line.includes('method="POST",route="/jobs",le'))
  .join('\n');
console.log(text.trimEnd());
```

Output

```
# HELP http_requests_total HTTP অনুরোধের সংখ্যা
# TYPE http_requests_total counter
http_requests_total{method="GET",route="/jobs",status="200"} 3
http_requests_total{method="POST",route="/jobs",status="201"} 1
http_requests_total{method="POST",route="/jobs",status="422"} 1
http_requests_total{method="GET",route="/jobs",status="500"} 1
# HELP http_request_duration_seconds অনুরোধের সময় (সেকেন্ড)
# TYPE http_request_duration_seconds histogram
```

```
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.005"} 1
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.01"} 1
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.05"} 2
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.1"} 2
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="0.5"} 3
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="1"} 3
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="5"} 4
http_request_duration_seconds_bucket{method="GET",route="/jobs",le="+Inf"} 4
http_request_duration_seconds_sum{method="GET",route="/jobs"} 2.043
http_request_duration_seconds_count{method="GET",route="/jobs"} 4
http_request_duration_seconds_sum{method="POST",route="/jobs"} 0.024
http_request_duration_seconds_count{method="POST",route="/jobs"} 2
```

Histogram-এর Bucket **ক্রমযোজিত**: `le="0.5"` মানে ০.৫ সেকেন্ড বা তার কম—চারটি GET-এর তিনটি। Label-এর মান সীমিত রাখুন: Route-এর নাম (`/jobs`), কখনো আসল Path (`/jobs/9381`) বা User ID নয়, নইলে প্রতিটি নতুন মান একটি নতুন Time Series তৈরি করে Metrics-ব্যবস্থার Memory শেষ করে (**High Cardinality**)।

Trace: একটি অনুরোধ Gateway থেকে Booking Service, সেখান থেকে Payment Service ও Database-এ গেলে, প্রতিটি ধাপ একটি **Span**, আর সব Span একটি Trace ID-তে যুক্ত। W3C-র `traceparent` Header এই ID এক Service থেকে অন্যটিতে বয়ে নেয়। OpenTelemetry হলো Trace, Metrics ও Log সংগ্রহের একটি মুক্ত মান, যা প্রায় সব পর্যবেক্ষণ-ব্যবস্থা বোঝে। DeployReady-এর `x-request-id` এর সবচেয়ে সরল রূপ: একটি ID, যা Response Header, Access Log ও Error Log—তিন জায়গায় একই।

২৮.১১ SLI, SLO ও Alerting

- **SLI** (Service Level Indicator): একটি মাপা সংখ্যা, যেমন সফল অনুরোধের অনুপাত, বা ৩০০ ms-এর কমে উত্তর পাওয়া অনুরোধের অনুপাত।
- **SLO** (Service Level Objective): লক্ষ্য, যেমন "৩০ দিনে ৯৯.৯% অনুরোধ সফল"।
- **SLA** (Service Level Agreement): গ্রাহকের সঙ্গে চুক্তি, ভঙ্গ হলে ক্ষতিপূরণ। SLA সাধারণত SLO-এর চেয়ে শিথিল।
- **Error Budget:** ১০০% থেকে SLO বাদ দিলে যা থাকে—৯৯.৯%-এ ০.১%। Budget বাকি থাকলে দ্রুত নতুন Feature ছাড়া যায়; শেষ হলে নির্ভরযোগ্যতার কাজ আগে।

slo-budget.mjs · JavaScript — চলবে: Node.js

```javascript
// SLO: ৩০ দিনে ৯৯.৯% অনুরোধ সফল। Error Budget = বাকি ০.১%।
const slo = 0.999;
const month = { total: 4_200_000, failed: 2_900 };
const lastHour = { total: 6_000, failed: 90 };

const sli = (w) => (w.total - w.failed) / w.total;
const budget = month.total * (1 - slo);
const pct = (x) => `${(x * 100).toFixed(3)}%`;
```

```
console.log('৩০ দিনের SLI:', pct(sli(month)));
console.log('Error Budget:', Math.round(budget), 'টি ব্যর্থ অনুরোধ');
console.log('খরচ হয়েছে:', pct(month.failed / budget));

// Burn Rate: এই হারে চললে Budget কত গুণ দ্রুত শেষ হবে।
const burnRate = (1 - sli(lastHour)) / (1 - slo);
console.log('শেষ ১ ঘণ্টার Error Rate:', pct(1 - sli(lastHour)));
console.log('Burn Rate:', burnRate.toFixed(1));
console.log('Budget শেষ হবে প্রায়', Math.round(30 / burnRate * 24), 'ঘণ্টায়');
console.log(burnRate >= 14.4 ? 'Page: এখনই কাউকে জাগান'
  : burnRate >= 1 ? 'Ticket: কাজের সময়ে দেখুন' : 'ঠিক আছে');
```

Output

```
৩০ দিনের SLI: 99.931%
Error Budget: 4200 টি ব্যর্থ অনুরোধ
খরচ হয়েছে: 69.048%
শেষ ১ ঘণ্টার Error Rate: 1.500%
Burn Rate: 15.0
Budget শেষ হবে প্রায় 48 ঘণ্টায়
Page: এখনই কাউকে জাগান
```

মাসের হিসাবে SLI ৯৯.৯৩১%—লক্ষ্যের ওপরে, তবু Budget-এর ৬৯% খরচ। আসল খবর শেষ ঘণ্টায়: Error Rate ১.৫%, যা অনুমোদিত হারের ১৫ গুণ (**Burn Rate**)। এই হারে পুরো মাসের Budget দুই দিনে শেষ। Google SRE Workbook-এর প্রচলিত নিয়মে এক ঘণ্টার Burn Rate ১৪.৪ বা বেশি হলে সঙ্গে সঙ্গে কাউকে জাগানো (Page), আর ধীর খরচে শুধু একটি Ticket। প্রতিটি একক Error-এ Page করলে Alert-ক্লান্তি আসে—মানুষ Alert উপেক্ষা করতে শেখে, আর আসল বিপদও উপেক্ষিত হয়। প্রতিটি Alert-এর সঙ্গে একটি Runbook থাকা উচিত: কী দেখতে হবে, কী করতে হবে।

২৮.১২ Secret, Supply Chain ও Infrastructure-এর নিরাপত্তা

- **Secret:** Code, Image, Frontend Bundle বা Log-এ নয়; Secret Store-এ, ন্যূনতম অনুমতিতে, নিয়মিত Rotation-সহ। স্বল্পমেয়াদি Credential ও Workload Identity (Platform নিজেই Service-কে পরিচয় দেয়) সবচেয়ে ভালো।
- **Supply Chain:** Lockfile, `npm ci`, Dependency Scan, Image Scan, এবং **SBOM** (Software Bill of Materials—Image-এ ঠিক কোন Package আছে তার তালিকা)। CI-এর Third-party Action Commit SHA দিয়ে Pin।
- **Network:** TLS সর্বত্র; Database শুধু Application-এর Network থেকে পৌঁছানো যায়; `/metrics` শুধু অভ্যন্তরীণ Network-এ।
- **Least Privilege:** CI-এর Token, Container-এর ব্যবহারকারী, Cloud-এর Role—প্রত্যেকে শুধু যা দরকার।

২৮.১৩ Backup, Disaster Recovery ও Incident Response

RPO (Recovery Point Objective): সর্বোচ্চ কতটা তথ্য হারানো সহনীয়—প্রতি ৬ ঘণ্টায় Backup হলে সবচেয়ে খারাপ ক্ষেত্রে ৬ ঘণ্টার তথ্য। **RTO** (Recovery Time Objective): কতক্ষণে সেবা ফিরতে হবে। দুটিই ব্যবসার সিদ্ধান্ত, এবং ছোট করতে খরচ বাড়ে (Continuous Backup, একাধিক Region)। অধ্যায় ২৬-এর শিক্ষা আবার: **যে Backup থেকে Restore করে দেখা হয়নি, তা Backup নয়।** নিয়মিত Restore Drill করুন এবং সময় মাপুন—সেটিই আপনার আসল RTO।

Incident Response-এর ধাপ: শনাক্ত (Alert বা গ্রাহক), দায়িত্ব নেওয়া (একজন Incident Lead), ক্ষতি থামানো (Rollback প্রায়ই সবচেয়ে দ্রুত), যোগাযোগ (Status Page, গ্রাহক), পুনরুদ্ধার, এবং শেষে **Postmortem**—দোষারোপহীন বিশ্লেষণ: কী ঘটেছিল, কেন ধরা পড়েনি, কী বদলাবে যাতে আবার না ঘটে।

২৮.১৪ Project: DeployReady Travel Operations API

DeployReady একটি ছোট API, যেখানে Operations দল কাজ (Job) তৈরি ও দেখে—যেমন "হোটেল Booking যাচাই"। Domain ইচ্ছাকৃতভাবে সরল; Project-এর আসল বিষয় তার চারপাশ: Configuration, Log, Metrics, Health, Shutdown, Container ও CI। Node.js-এর অন্তর্নির্মিত `http` দিয়ে লেখা, কোনো npm Package নেই। **এটি আসল Booking বা Payment Backend নয়;** Data Memory-তে থাকে।

File-এর কাঠামো

Folder-এর কাঠামো

```
deploy-ready/
├── package.json          Script: start, test, check, smoke
├── README.md             চালানো, Environment Variable, সীমাবদ্ধতা
├── main.mjs              Config যাচাই, Signal ধরা, Server চালু
├── healthcheck.mjs       Docker HEALTHCHECK
├── Dockerfile
├── .dockerignore
├── compose.yaml
├── .env.example          .env-এর নমুনা (আসল .env Git-এ নয়)
├── .github/workflows/ci.yml
├── scripts/
│   ├── check-deploy.mjs Deploy File-এর স্থির যাচাই
│   └── smoke.mjs         Deploy-এর পরের Smoke Test
├── src/
│   ├── config.mjs        Configuration ও Fail Fast
│   ├── logger.mjs        Structured Log
│   ├── metrics.mjs       Prometheus Format
│   ├── jobs.mjs          Domain ও Memory Store
│   ├── app.mjs           Route, Request ID, Error চুক্তি
│   └── server.mjs        জীবনচক্র ও Graceful Shutdown
└── tests/                ১৪টি Test
```

API

Method	Path	Auth	ফল
GET	`/health/live`	না	200 যতক্ষণ Process চলছে
GET	`/health/ready`	না	200; Shutdown বা Dependency সমস্যায় 503
GET	`/version`	না	Version ও Commit
GET	`/metrics`	Token	Prometheus Text Format
GET	`/api/jobs`	না	Job-এর তালিকা
POST	`/api/jobs`	Token	201; ভুল Input-এ 422

src/jobs.mjs

Domain ও একটি Memory Storeı Store-এর Method-গুলো `async`, যাতে পরে Database বসালে বাকি Code বদলাতে না হয়:

jobs.mjs · JavaScript — চলবে: Node.js

```
// Domain: Travel Operations-এর কাজ (যেমন "হোটেল Booking যাচাই")।
// Storage Interface async, যাতে পরে Database বসানো যায় (অধ্যায় ২৬)।
export const PRIORITIES = ['low', 'normal', 'high'];

export class ValidationError extends Error {
  constructor(details) {
    super('Input যাচাইয়ে ব্যর্থ');
    this.details = details;
  }
}

export function validateJob(input) {
  const details = [];
  const title = typeof input?.title === 'string' ? input.title.trim() : '';
  const priority = input?.priority ?? 'normal';
  if (title.length < 3 || title.length > 120) {
    details.push('title: ৩ থেকে ১২০ অক্ষর');
  }
  if (!PRIORITIES.includes(priority)) {
    details.push(`priority: ${PRIORITIES.join(', ')}`);
  }
  if (details.length) throw new ValidationError(details);
  return { title, priority };
}

export function createMemoryJobStore({ now = () => new Date() } = {}) {
  const jobs = [];
  return {
```

```
    async list() {
      return jobs.map((job) => ({ ...job }));
    },
    async create(input) {
      const job = { id: jobs.length + 1, ...validateJob(input),
        status: 'open', createdAt: now().toISOString() };
      jobs.push(job);
      return { ...job };
    },
    async ping() {
      return true;
    },
  };
}
```

src/app.mjs

প্রথম অংশ: নিজস্ব HTTP Error, Route-এর তালিকা এবং JSON পড়া—Content-Type যাচাই (415), আকারের সীমা (413) ও ভাঙা JSON (400):

app.mjs · JavaScript অংশ ১ / ৫ (Part 1 of 5) চলবে: Node.js

```
import { randomUUID } from 'node:crypto';
import { ValidationError } from './jobs.mjs';

class HttpError extends Error {
  constructor(status, code, message) {
    super(message);
    this.status = status;
    this.code = code;
  }
}

const ROUTES = {
  'GET /health/live': 'live',
  'GET /health/ready': 'ready',
  'GET /version': 'version',
  'GET /metrics': 'metrics',
  'GET /api/jobs': 'listJobs',
  'POST /api/jobs': 'createJob',
};

async function readJson(req, limit = 8 * 1024) {
  if (!/^application\/json\b/.test(req.headers['content-type'] ?? '')) {
    throw new HttpError(415, 'UNSUPPORTED_MEDIA_TYPE', 'JSON পাঠান');
  }
  let size = 0;
  const chunks = [];
```

```
  for await (const chunk of req) {
    size += chunk.length;
    if (size > limit) throw new HttpError(413, 'TOO_LARGE', 'Body খুব বড়');
    chunks.push(chunk);
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString('utf8'));
  } catch {
    throw new HttpError(400, 'BAD_JSON', 'অবৈধ JSON');
  }
}

// ...
```

Dependency যাচাইয়ের সময়সীমা, এবং Token যাচাই। Development-এ `OPS_TOKEN` না থাকলে যাচাই হয় না; Production-এ Config নিজেই Token বাধ্যতামূলক করে:

app.mjs · JavaScript অংশ ২ / ৫ (Part 2 of 5) চলবে: Node.js

```
// ...

function withTimeout(promise, ms) {
  let timer;
  const timeout = new Promise((resolve, reject) => {
    timer = setTimeout(() => reject(new Error('timeout')), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

export function createApp({ config, logger, metrics, store,
  readyTimeoutMs = 1000 }) {
  const state = { draining: false, inFlight: 0 };

  const requireOps = (req) => {
    if (!config.opsToken) return; // শুধু development-এ Token ছাড়া
    if (req.headers.authorization !== `Bearer ${config.opsToken}`) {
      throw new HttpError(401, 'UNAUTHORIZED', 'Token প্রয়োজন');
    }
  };

// ...
```

প্রতিটি Route-এর কাজ। Readiness-এ Dependency (`store.ping()`) এক সেকেন্ডের বেশি অপেক্ষা করলে 503:

app.mjs · JavaScript অংশ ৩ / ৫ (Part 3 of 5) চলবে: Node.js

```
// ...

  const handlers = {
```

```
    live: async () => [200, { status: 'live' }],
    ready: async () => {
      if (state.draining) return [503, { status: 'draining' }];
      try {
        await withTimeout(store.ping(), readyTimeoutMs);
        return [200, { status: 'ready' }];
      } catch {
        return [503, { status: 'dependency-unavailable' }];
      }
    },
    version: async () => [200, { version: config.version,
      commit: config.commit }],
    metrics: async (req) => {
      requireOps(req);
      metrics.setGauge('http_in_flight_requests', state.inFlight);
      return [200, metrics.render(), 'text/plain; version=0.0.4'];
    },
    listJobs: async () => [200, { data: await store.list() }],
    createJob: async (req) => {
      requireOps(req);
      return [201, { data: await store.create(await readJson(req)) }];
    },
  };

// ...
```

প্রতিটি অনুরোধের সাধারণ কাজ: Request ID (Client-এর পাঠানোটি নিরাপদ হলে তা-ই, নইলে নতুন), Response শেষ হলে Access Log ও Metrics:

app.mjs · JavaScript অংশ 8 / ৫ (Part 4 of 5) চলবে: Node.js

```
// ...

  async function handle(req, res) {
    const started = process.hrtime.bigint();
    const incoming = req.headers['x-request-id'];
    const requestId = /^[\w-]{8,64}$/.test(incoming ?? '')
      ? incoming : randomUUID();
    const { pathname } = new URL(req.url, 'http://localhost');
    const name = ROUTES[`${req.method} ${pathname}`];
    // Metrics-এর Label-এ আসল Path নয়, Route-এর নাম: সীমিত সংখ্যক মান।
    const route = name ? pathname : 'unmatched';
    state.inFlight += 1;
    res.setHeader('x-request-id', requestId);
    res.on('finish', () => {
      state.inFlight -= 1;
      const seconds = Number(process.hrtime.bigint() - started) / 1e9;
      metrics.observeRequest({ method: req.method, route,
        status: res.statusCode, seconds });
```

```
      logger.info('http.request', { requestId, method: req.method, route,
        status: res.statusCode, durationMs: Math.round(seconds * 1000) });
    });
    try {
      if (!name) throw new HttpError(404, 'NOT_FOUND', 'Route পাওয়া যায়নি');
      const [status, body, type] = await handlers[name](req);
      send(res, status, body, type);
    } catch (error) {
      sendError(res, error, requestId);
    }
  }

// ...
```

Error চুক্তি: যাচাইয়ের ভুলে 422 ও কারণের তালিকা; জানা HTTP Error-এ তার নিজের Status; আর অপ্রত্যাশিত Error-এ Client পায় শুধু একটি সাধারণ বার্তা ও Request ID, পুরো Stack যায় Log-এ:

app.mjs · JavaScript অংশ ৫ / ৫ (Part 5 of 5) চলবে: Node.js

```
// ...

  function sendError(res, error, requestId) {
    if (error instanceof ValidationError) {
      return send(res, 422, { error: { code: 'VALIDATION',
        message: error.message, details: error.details } });
    }
    if (error instanceof HttpError) {
      return send(res, error.status, { error: { code: error.code,
        message: error.message } });
    }
    logger.error('http.unhandled', { requestId, message: error.message,
      stack: error.stack });
    return send(res, 500, { error: { code: 'INTERNAL',
      message: 'Server-এ সমস্যা', requestId } });
  }

  return { handle, state };
}

function send(res, status, body, type = 'application/json; charset=utf-8') {
  res.writeHead(status, { 'Content-Type': type });
  res.end(typeof body === 'string' ? body : JSON.stringify(body));
}
```

গ্রাহক যখন বলেন "Error এসেছিল", তিনি Request ID দিতে পারেন, আর Operator সেই ID দিয়ে Log-এ পুরো ঘটনা খুঁজে পান—Stack Trace গ্রাহকের Screen-এ না দেখিয়েই।

main.mjs

main.mjs · JavaScript — চলবে: Node.js

```
import { loadConfig, describeConfig } from './src/config.mjs';
import { createLogger } from './src/logger.mjs';
import { createMetrics } from './src/metrics.mjs';
import { createMemoryJobStore } from './src/jobs.mjs';
import { startServer } from './src/server.mjs';

let config;
try {
  config = loadConfig();
} catch (error) {
  console.error(error.message);
  process.exit(1); // ভুল Config নিয়ে কখনো চালু নয়
}

const logger = createLogger({ level: config.logLevel,
  base: { service: 'deploy-ready', version: config.version } });
logger.info('config.loaded', describeConfig(config));
const { shutdown } = await startServer({ config, logger,
  metrics: createMetrics(), store: createMemoryJobStore() });

// Container Platform বন্ধ করার আগে SIGTERM পাঠায়; Ctrl+C পাঠায় SIGINT।
for (const signal of ['SIGTERM', 'SIGINT']) {
  process.once(signal, async () => {
    const { forced } = await shutdown(signal);
    process.exit(forced ? 1 : 0);
  });
}
process.on('unhandledRejection', async (reason) => {
  logger.error('process.unhandledRejection', { message: String(reason) });
  await shutdown('unhandledRejection');
  process.exit(1);
});
```

`main.mjs` এই Listing থেকে চালানো হয়নি, কারণ এটি থামে না (একটি Server); Project-এর Test একে ভুল Config দিয়ে চালিয়ে দেখে যে Exit Code 1 এবং stdout ফাঁকা, আর QA-র Deploy যাচাই একে Production Mode-এ চালিয়ে Smoke Test করে।

scripts/smoke.mjs

Deploy-এর পরে, Traffic দেওয়ার আগে, পাঁচটি মৌলিক যাচাই। একটিও ব্যর্থ হলে Exit Code 1, যাতে Pipeline Rollback করতে পারে:

smoke.mjs · JavaScript — চলবে: Node.js

```
// Deploy-এর পরে Smoke Test: node scripts/smoke.mjs https://staging.example.com
```

```
// কোনো ধাপ ব্যর্থ হলে Exit Code 1, যাতে Pipeline থেমে Rollback করতে পারে।
const base = process.argv[2] ?? 'http://127.0.0.1:3000';
const token = process.env.OPS_TOKEN;
const auth = token ? { Authorization: `Bearer ${token}` } : {};

const checks = [
  ['liveness', () => fetch(`${base}/health/live`), 200],
  ['readiness', () => fetch(`${base}/health/ready`), 200],
  ['version', () => fetch(`${base}/version`), 200],
  ['create job', () => fetch(`${base}/api/jobs`, { method: 'POST',
    headers: { 'Content-Type': 'application/json', ...auth },
    body: JSON.stringify({ title: 'Smoke test job', priority: 'low' }) }), 201],
  ['list jobs', () => fetch(`${base}/api/jobs`), 200],
];

let failed = 0;
for (const [name, run, expected] of checks) {
  try {
    const res = await run();
    const ok = res.status === expected;
    if (!ok) failed += 1;
    console.log(`${ok ? 'PASS' : 'FAIL'} ${name}: ${res.status}`);
  } catch (error) {
    failed += 1;
    console.log(`FAIL ${name}: ${error.message}`);
  }
}
process.exit(failed ? 1 : 0);
```

Test: Graceful Shutdown

Shutdown-এর আচরণ শুধু পড়ে বিশ্বাস করা কঠিন, তাই Test একটি "ধীর Upload" তৈরি করে—Header ও Body-র কিছু অংশ পাঠিয়ে থেমে থাকা একটি অনুরোধ—এবং তার মাঝখানে Shutdown শুরু করে:

shutdown.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('Drain: Readiness 503, চলমান অনুরোধ সফলভাবে শেষ', async () => {
  const app = await start({ env: { DRAIN_DELAY_MS: '200' } });
  const upload = slowUpload(app.port);
  await new Promise((resolve) => setTimeout(resolve, 50));
  const done = app.shutdown('SIGTERM');
  const ready = await call(app.base, '/health/ready');
  assert.deepEqual([ready.status, ready.body], [503, { status: 'draining' }]);
  upload.finish();
  assert.equal(await upload.response, 201);
  assert.deepEqual(await done, { forced: false });
  assert.equal(app.shutdown('again'), done); // দ্বিতীয় Signal-এ একই কাজ
```

```
  const events = app.lines.map((l) => l.event);
  assert.ok(events.indexOf('shutdown.started')
    < events.indexOf('shutdown.complete'));
});

test('Timeout: শেষ না হওয়া অনুরোধ জোর করে বন্ধ', async () => {
  const app = await start({ env: { SHUTDOWN_TIMEOUT_MS: '150' } });
  const upload = slowUpload(app.port);
  upload.response.catch(() => {}); // সংযোগ কাটা প্রত্যাশিত
  await new Promise((resolve) => setTimeout(resolve, 50));
  const started = Date.now();
  assert.deepEqual(await app.shutdown('SIGTERM'), { forced: true });
  assert.ok(Date.now() - started >= 140);
  await assert.rejects(upload.response);
});
```

প্রথম Test প্রমাণ করে: Shutdown শুরুর পর Readiness 503, অথচ চলমান Upload সফলভাবে 201 পায় এবং Shutdown `forced: false` দিয়ে শেষ হয়। দ্বিতীয়টি প্রমাণ করে: Upload কখনো শেষ না হলে ১৫০ ms পর সংযোগ জোর করে বন্ধ এবং `forced: true`। Test লেখার সময় একটি শিক্ষণীয় ভুল ধরা পড়েছিল: Health Check Script `spawnSync` দিয়ে চালালে Test নিজেই ব্যর্থ হতো, কারণ Synchronous Spawn একই Process-এর Event Loop আটকে রাখে, আর সেই Process-এর Server উত্তর দিতে পারে না। সমাধান: Asynchronous `spawn`।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd deploy-ready
npm test
npm run check
npm start
```

`npm test` ১৪টি Test চালায়: `config` ৪ (Default, সব ভুল একসঙ্গে, Production-এর Token, ভুল Config-এ Process-এর Exit Code 1), `observability` ২ (Log-এর গঠন ও লুকানো মান, Prometheus Format), `app` ৫ (Health ও Request ID, Input-এর প্রতিটি ভুল, Metrics-এর অনুমতি ও Label, অপ্রত্যাশিত Error, Dependency-র Timeout), এবং `shutdown` ৩ (Drain, জোর করে বন্ধ, HEALTHCHECK Script):

Output

```
# tests 14
# pass 14
# fail 0
```

QA-তে আরও যা চালানো হয়েছে: `compose.yaml` ও `ci.yml` দুটি YAML Parser (`yaml` 2.8.3 ও `js-yaml` 3.14.2) দিয়ে Parse—দুটির ফল হুবহু এক; Compose-এর Port, Secret, সময়ের হিসাব ও Hardening; CI-এর অনুমতি, Job-এর নির্ভরতা ও Timeout, এবং Workflow-এর ডাকা প্রতিটি npm Script ও File-এর অস্তিত্ব। এরপর `main.mjs`

Production Mode-এ চালু করে Smoke Test—পাঁচটিই পাস, আর Token ছাড়া চালালে Smoke Test ঠিকভাবে ব্যর্থ (Exit Code 1); Startup Log-এ Token-এর বদলে `[REDACTED]`। **চালানো হয়নি:** `docker build`, `docker compose up` (Docker Daemon চালু ছিল না) এবং GitHub-এ Workflow।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Process চালুর সঙ্গে সঙ্গে Exit Code 1	Configuration ভুল	stderr-এ "Configuration ভুল"-এর তালিকা
Container বারবার Restart	Liveness ব্যর্থ, বা Process Crash	`docker logs`, HEALTHCHECK-এর Port
Deploy-এর সময় কিছু অনুরোধে 502	Readiness-এর আগেই Traffic, বা Drain নেই	`DRAIN_DELAY_MS`, Readiness Probe
Container বন্ধ হতে ঠিক ১০ বা ৩০ সেকেন্ড লাগে, তারপর হঠাৎ	SIGTERM Application-এ পৌঁছায় না	Exec Form CMD, `npm start` নয়
`/metrics`-এ 401	Token নেই বা ভুল	`Authorization: Bearer <OPS_TOKEN>`
Metrics-ব্যবস্থার Memory বাড়ছে	Label-এ আসল Path বা ID	Route-এর নাম Label-এ
Windows-এ বানানো Image Linux-এ চলে না	Host-এর `node_modules` কপি	`.dockerignore`, Image-এ `npm ci`

২৮.১৫ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
`.env` বা Private Key Git-এ	ইতিহাসে চিরকাল থেকে যায়	`.gitignore`, Secret Store, ফাঁস হলে Rotation
শুধু `latest` Tag	কোনটি চলছে, কোনটিতে ফিরব—অজানা	Commit SHA Tag ও Image Digest
Liveness-এ Database যাচাই	Database বন্ধ হলে সব Container Restart	Liveness সরল, Dependency Readiness-এ
Graceful Shutdown নেই	প্রতিটি Deploy-এ কিছু অনুরোধ কাটে	Drain, `server.close`, Timeout
`CMD npm start`	SIGTERM পৌঁছায় না	`CMD ["node", "main.mjs"]`
Root হিসেবে Container	আক্রমণকারী বেশি ক্ষমতা পায়	`USER node`

ভুল	কেন সমস্যা	সমাধান
শুধু HTTP 200 দেখে "সব ঠিক"	ধীরগতি বা আংশিক ব্যর্থতা চোখে পড়ে না	Latency Histogram, Error Rate, SLO
প্রতিটি Error-এ Page	Alert-ক্লান্তি	Burn Rate-ভিত্তিক Alert, Runbook
Memory-তে Session নিয়ে একাধিক Replica	Instance বদলালে ব্যবহারকারী Logout	Session Store (Database বা Redis)
Backup আছে, Restore কখনো চেষ্টা হয়নি	দরকারের দিন অকেজো	নিয়মিত Restore Drill

২৮.১৬ Debugging নির্দেশিকা

১. **প্রথমে সংস্করণ:** `/version` থেকে কোন Commit চলছে নিশ্চিত হন; সমস্যা কি নতুন Deploy-এর পরে শুরু?
২. **Request ID ধরে খুঁজুন:** গ্রাহক বা Response Header থেকে ID নিয়ে Log-এ সেই ID-র সব Line।
৩. **Metrics-এ প্রবণতা:** Error Rate ও p95 কখন থেকে বেড়েছে; কোন Route-এ; Traffic কি স্বাভাবিকের চেয়ে বেশি (Saturation)?
৪. **Health আলাদা করে দেখুন:** Live 200 কিন্তু Ready 503 মানে Process ঠিক আছে, Dependency বা Drain-এ সমস্যা।
৫. **Container-এ:** `docker ps` (Health অবস্থা), `docker logs`, `docker inspect` (Exit Code, OOMKilled)।
৬. **Local-এ পুনরুৎপাদন:** একই Image Digest ও একই Environment Variable (গোপন মান ছাড়া) দিয়ে চালান।
৭. **সন্দেহ হলে Rollback, পরে তদন্ত:** গ্রাহকের ক্ষতি থামানো আগে; কারণ খোঁজা পরে, ঠান্ডা মাথায়।

২৮.১৭ পেশাদার অভ্যাস

- **একবার Build, সবখানে Deploy:** Staging ও Production-এ একই Image Digest।
- **Configuration শুরুতেই যাচাই, গোপন মান কখনো Log-এ নয়।**
- **প্রতিটি Release-এর পরিচয়:** Commit SHA Log, Metrics ও `/version`-এ।
- **Health দুই ভাগে:** Liveness সরল, Readiness সত্যিকারের প্রস্তুতি।
- **Shutdown-এর সময়ের হিসাব লিখে রাখুন:** Drain + Timeout < Grace Period।
- **SLO আগে, Alert পরে:** ব্যবহারকারীর অভিজ্ঞতা মাপুন, CPU নয়।
- **প্রতিটি Alert-এর Runbook, প্রতিটি Incident-এর Postmortem।**
- **যা যাচাই হয়নি, তা স্পষ্ট লিখুন:** এই অধ্যায়ের README যেমন বলে Docker Build চালানো হয়নি।

২৮.১৮ অনুশীলনী

প্রাথমিক স্তর

১. Development ও Production পরিবেশের পাঁচটি পার্থক্য লিখুন, প্রতিটির জন্য একটি করে Production-এর ব্যবস্থা।

২. `PORT=abc` হলে Server-এর চালু হওয়া কেন ভুল? DeployReady এ ক্ষেত্রে কী করে?

৩. Image ও Container-এর পার্থক্য লিখুন। Dockerfile-এ `package.json` কেন Code-এর আগে কপি করা হয়?

৪. `.dockerignore`-এ `node_modules`, `.env` ও `.git` রাখার কারণ লিখুন।

মধ্যম স্তর

৫. তিনটি পরিস্থিতিতে Liveness ও Readiness কী উত্তর দেবে: Database বন্ধ; Shutdown শুরু; Event Loop পুরো আটকে গেছে।

৬. `node:test` দিয়ে লিখুন: ফাঁকা, শুধু Space-এর ও দুই অক্ষরের `title`-এ 422 এবং কোনো Job তৈরি হয় না; `title` না দিলেও 422।

৭. একটি Log File থেকে নির্দিষ্ট Request ID-র সব ঘটনা ক্রমে বের করার Function লিখুন; ভাঙা Line উপেক্ষা করুন।

৮. Rolling, Blue/Green ও Canary তুলনা করুন: কোনটি কখন বেছে নেবেন?

চ্যালেঞ্জ স্তর

৯. ৪,০০০ অনুরোধের ৩,৯৮০টি সফল। Success Rate, ৯৯.৯% SLO-তে অনুমোদিত ব্যর্থতা, Budget-এর খরচ ও Burn Rate হিসাব করুন।

১০. DeployReady-তে PostgreSQL যোগ করার পরিকল্পনা লিখুন: Store, Migration, Readiness, Secret, Test ও Rollback।

১১. একটি Readiness যাচাই লিখুন যা Dependency-কে সময়সীমা দেয় এবং ফল দুই সেকেন্ড Cache করে।

১২. এই অধ্যায়ের CI Workflow-কে আরও শক্ত করার অন্তত পাঁচটি প্রস্তাব দিন।

২৮.১৯ অনুশীলনীর সমাধান

সমাধান ১ — (১) Configuration: Local-এ `.env`; Production-এ Platform-এর Environment ও Secret Store, শুরুতেই যাচাই। (২) Process: Terminal; Production-এ Container Platform, HEALTHCHECK ও স্বয়ংক্রিয় Restart। (৩) Data: Memory; Production-এ টেকসই Database, Backup ও Restore Drill। (৪) Network: `localhost`; Production-এ TLS, Load Balancer, শুধু প্রয়োজনীয় Port খোলা। (৫) ব্যর্থতা জানা: Developer চোখে দেখেন; Production-এ Structured Log, Metrics, SLO-ভিত্তিক Alert ও Runbook।

সমাধান ২ — `Number('abc')` হলো `NaN`; Node.js-এ `listen(NaN)` একটি এলোমেলো খালি Port বেছে নিতে পারে, অথবা অন্য অপ্রত্যাশিত আচরণ—Server "চালু", কিন্তু Load Balancer যে Port-এ পাঠায় সেখানে কেউ নেই, এবং কেউ কারণ বোঝে না। DeployReady-এর `loadConfig` `PORT`-কে শুধু অঙ্কের String হিসেবে ও ০–৬৫৫৩৫ সীমায় গ্রহণ করে; নইলে সব ভুলের তালিকা stderr-এ লিখে Exit Code 1 (উদাহরণ ০১ ও `config.test.mjs`-এর শেষ Test)। Platform তখন Deploy ব্যর্থ দেখায় এবং পুরোনো সংস্করণ চলতে থাকে।

সমাধান ৩ — Image একটি অপরিবর্তনীয় Template (স্তরে সাজানো File System ও চালানোর নির্দেশ); Container তার একটি চলমান Instance, নিজস্ব Process ও লেখার স্তরসহ। Docker প্রতিটি নির্দেশের স্তর Cache করে, আর কোনো স্তর বদলালে তার পরের সব আবার তৈরি হয়। `package.json` (ও Lockfile) কম বদলায়, Code প্রায় প্রতিবার; তাই আগে `package.json` ও `npm ci`, পরে Code কপি করলে Dependency Install-এর ধীর স্তর Cache থেকে আসে।

সমাধান ৪ — `node_modules`: Host-এর Native Module (যেমন Windows-এ Build করা) Linux Container-এ চলে না, আর Build Context বিশাল হয়; Image-এ `npm ci` করে নতুন করে Install করুন। `.env`: গোপন মান Image-এ ঢুকে যায়, আর Image Registry-তে গেলে যে-কেউ Pull করে পড়তে পারে। `.git`: পুরো ইতিহাস, পুরোনো Commit-এ ভুলে রাখা Secret-সহ, Image-এ চলে যায়, এবং আকার বাড়ে।

সমাধান ৫

পরিস্থিতি	Liveness	Readiness	কেন
Database বন্ধ	200	503	Restart করে লাভ নেই; শুধু Traffic সরানো
Shutdown শুরু	200	503	চলমান কাজ শেষ হোক, নতুন না আসুক
Event Loop আটকে	উত্তর নেই (Timeout)	উত্তর নেই	Process অকেজো; Liveness ব্যর্থ, তাই Restart

তৃতীয় ক্ষেত্রেই Liveness-এর আসল মূল্য: একটি অসীম Loop বা Synchronous ভারী কাজে Process জীবিত কিন্তু অকেজো, আর শুধু Restart-ই তাকে ফেরাতে পারে।

সমাধান ৬

empty-title.test.mjs · JavaScript — চলবে: Node.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { start, call, json, TOKEN }
  from '../../project/deploy-ready/tests/helpers.mjs';

test('ফাঁকা বা শুধু Space-এর title: 422 এবং কারণ', async (t) => {
  const app = await start();
  t.after(() => app.shutdown('test'));
```

```
  for (const title of ['', '   ', 'ab']) {
    const res = await call(app.base, '/api/jobs', { method: 'POST',
      token: TOKEN, ...json({ title }) });
    assert.equal(res.status, 422);
    assert.deepEqual(res.body.error.details, ['title: ৩ থেকে ১২০ অক্ষর']);
  }
  const list = await call(app.base, '/api/jobs');
  assert.deepEqual(list.body.data, []);
});

test('title না দিলেও 422, Server ভাঙে না', async (t) => {
  const app = await start();
  t.after(() => app.shutdown('test'));
  const res = await call(app.base, '/api/jobs', { method: 'POST',
    token: TOKEN, ...json({ priority: 'low' }) });
  assert.equal(res.status, 422);
});
```

Output

```
ok 1 - ফাঁকা বা শুধু Space-এর title: 422 এবং কারণ
ok 2 - title না দিলেও 422, Server ভাঙে না
# tests 2
# pass 2
# fail 0
```

শুধু Status নয়, কারণের তালিকা এবং "কোনো Job তৈরি হয়নি"—দুটিই যাচাই করা হয়। Test Project-এর `helpers.mjs` আবার ব্যবহার করে, তাই আসল Server Port 0-তে চালু হয়।

সমাধান ৭

trace-request.mjs · JavaScript — চলবে: Node.js

```
// একজন গ্রাহক বলছেন "Error এসেছিল, Request ID: req-b2"। Log থেকে খুঁজে বের করা।
// নিচের Text একটি Log File-এর মতো: প্রতি Line একটি JSON, একটি Line ভাঙা।
const t = (ms) => `2026-10-18T09:00:00.0${ms}Z`;
const logText = [
  { time: t(10), level: 'info', event: 'http.request', requestId: 'req-a1',
    status: 200, durationMs: 12 },
  { time: t(20), level: 'warn', event: 'db.slow', requestId: 'req-b2',
    durationMs: 4100 },
  { time: t(30), level: 'error', event: 'http.unhandled',
    requestId: 'req-b2', message: 'pool exhausted' },
  'not json at all',
  { time: t(40), level: 'info', event: 'http.request', requestId: 'req-b2',
    status: 500, durationMs: 4210 },
].map((e) => (typeof e === 'string' ? e : JSON.stringify(e))).join('\n');
```

```
function trace(text, requestId) {
  return text.split('\n').flatMap((line) => {
    try {
      const entry = JSON.parse(line);
      return entry.requestId === requestId ? [entry] : [];
    } catch {
      return []; // Structured নয় এমন Line বাদ
    }
  });
}

for (const entry of trace(logText, 'req-b2')) {
  const extra = entry.message ?? (entry.status
    ? `${entry.status} in ${entry.durationMs} ms` : `${entry.durationMs} ms`);
  console.log(`${entry.time.slice(11, 23)} ${entry.level.padEnd(5)}`,
    `${entry.event}: ${extra}`);
}
```

Output

```
09:00:00.020 warn  db.slow: 4100 ms
09:00:00.030 error http.unhandled: pool exhausted
09:00:00.040 info  http.request: 500 in 4210 ms
```

তিনটি Line মিলে গল্পটি স্পষ্ট: Database ধীর হলো, Connection Pool শেষ হলো, এবং অনুরোধটি চার সেকেন্ডের বেশি পরে 500 দিল। Request ID ছাড়া এই তিনটি Line হাজারো Line-এর মধ্যে আলাদা করা প্রায় অসম্ভব।

সমাধান ৮ — **Rolling:** অতিরিক্ত Infrastructure নেই, ছোট ও পশ্চাৎ-সামঞ্জস্যপূর্ণ পরিবর্তনে উপযুক্ত; কিন্তু কিছুক্ষণ দুই সংস্করণ একসঙ্গে চলে, আর Rollback-ও আরেকটি Rolling Deploy। **Blue/Green:** বড় বা ঝুঁকিপূর্ণ Release, যেখানে তাৎক্ষণিক Rollback চাই এবং দ্বিগুণ পরিবেশের খরচ বহন করা যায়; Database উভয় পরিবেশে এক, তাই Migration সামঞ্জস্যপূর্ণ হতেই হবে। **Canary:** অনেক Traffic ও ভালো Metrics থাকলে সেরা—আসল ব্যবহারকারীর ছোট অংশে ঝুঁকি সীমিত রেখে সংখ্যা দিয়ে সিদ্ধান্ত (উদাহরণ ০৫); কিন্তু অল্প Traffic-এ পরিসংখ্যান দুর্বল।

সমাধান ৯

success-rate.mjs · JavaScript — চলবে: Node.js

```
// 8,000 অনুরোধে ৩,৯৮০ সফল। SLO ৯৯.৯% হলে কী দাঁড়ায়?
const total = 4000;
const ok = 3980;
const slo = 0.999;

const successRate = ok / total;
const allowedFailures = total * (1 - slo);
const failures = total - ok;

console.log('Success Rate:', `${(successRate * 100).toFixed(1)}%`);
console.log('SLO অনুযায়ী অনুমোদিত ব্যর্থতা:', allowedFailures.toFixed(1));
console.log('আসল ব্যর্থতা:', failures);
console.log('Budget-এর', `${((failures / allowedFailures) * 100).toFixed(0)}%`,
  'খরচ; Burn Rate', ((1 - successRate) / (1 - slo)).toFixed(1));
console.log(successRate >= slo ? 'SLO পূরণ' : 'SLO পূরণ হয়নি');
```

Output

```
Success Rate: 99.5%
SLO অনুযায়ী অনুমোদিত ব্যর্থতা: 4.0
আসল ব্যর্থতা: 20
Budget-এর 500% খরচ; Burn Rate 5.0
SLO পূরণ হয়নি
```

8,000 অনুরোধে ৯৯.৯% মানে মাত্র ৪টি ব্যর্থতার অনুমতি; ২০টি হয়েছে—Budget পাঁচ গুণ খরচ। ৯৯.৫% শুনতে ভালো, কিন্তু SLO-র প্রেক্ষিতে খারাপ। আর ছোট Window-তে (8,000 অনুরোধ) একটি খারাপ মিনিটই ফল বদলে দেয়; তাই SLO সাধারণত ২৮ বা ৩০ দিনের Window-তে মাপা হয়।

সমাধান ১০ — (১) **Store:** `createPostgresJobStore(pool)` একই `list`, `create`, `ping` চুক্তিতে (অধ্যায় ২৬-এর Adapter-এর মতো); `ping` চালায় `SELECT 1`। (২) **Migration:** `001-jobs.sql` (Table, CHECK Constraint, Index); Deploy-এর আগে আলাদা ধাপে চলে, Application চালুর সময় নয়; প্রতিটি পশ্চাৎ-সামঞ্জস্যপূর্ণ (Expand/Contract)। (৩) **Readiness:** `store.ping()` ইতিমধ্যে সময়সীমায় যাচাই হয়, তাই Database বন্ধ হলে 503। (৪) **Secret:** `DATABASE_URL` Secret Store থেকে; `loadConfig`-এ Production-এ বাধ্যতামূলক, আর `describeConfig`-এ লুকানো। (৫) **Test:** Unit Test Memory Store দিয়ে; Integration Test একটি আসল PostgreSQL-এ (CI-তে Service Container)। (৬) **Shutdown:** `server.close`-এর পরে `pool.end()`। (৭) **Rollback:** Application-এর আগের Image চলতে পারে কারণ Migration শুধু যোগ করেছে; প্রথম Deploy-এর আগে Backup ও Restore Drill।

সমাধান ১১

cached-readiness.mjs · JavaScript — চলবে: Node.js

```
// Readiness: Dependency-র যাচাই সময়সীমিত, আর ফল কিছুক্ষণ Cache।
// প্রতি সেকেন্ডে আসা Probe যেন Database-কে নিজেই চাপে না ফেলে।
export function createReadiness(check, { timeoutMs = 500, cacheMs = 2000,
  now = Date.now } = {}) {
  let cached = null;
  let calls = 0;
  async function probe() {
    if (cached && now() - cached.at < cacheMs) return cached.ready;
    calls += 1;
    let timer;
    const timeout = new Promise((resolve) => {
      timer = setTimeout(() => resolve(false), timeoutMs);
    });
    const ready = await Promise.race([
      check().then(() => true, () => false), timeout,
    ]).finally(() => clearTimeout(timer));
    cached = { ready, at: now() };
    return ready;
  }
  return { probe, calls: () => calls };
}

let clock = 0;
let healthy = true;
const db = () => (healthy ? Promise.resolve()
  : new Promise(() => {})); // অসুস্থ Database: কখনো উত্তর দেয় না
const readiness = createReadiness(db, { timeoutMs: 50, now: () => clock });

console.log('t=0:', await readiness.probe());
clock = 1000;
healthy = false;
console.log('t=1s (Cache থেকে):', await readiness.probe());
clock = 2500;
console.log('t=2.5s (নতুন যাচাই, Timeout):', await readiness.probe());
console.log('আসল যাচাই হয়েছে', readiness.calls(), 'বার');
```

Output

```
t=0: true
t=1s (Cache থেকে): true
t=2.5s (নতুন যাচাই, Timeout): false
আসল যাচাই হয়েছে 2 বার
```

এক সেকেন্ডে Database অসুস্থ হলেও Cache-এর কারণে Readiness তখনো `true` —এটি ইচ্ছাকৃত মূল্য: সর্বোচ্চ দুই সেকেন্ড দেরিতে জানা, বিনিময়ে প্রতি Probe-এ Database-এ একটি Query না পাঠানো। অসুস্থ Database কখনো উত্তর দেয় না, তাই Timeout ছাড়া Probe নিজেই আটকে যেত। `clock` Inject করা, যাতে Test-এ সত্যিকারের অপেক্ষা লাগে না।

সমাধান ১২ — (১) প্রতিটি Action পূর্ণ Commit SHA দিয়ে Pin, এবং Dependabot বা অনুরূপ ব্যবস্থায় হালনাগাদ। (২) `npm ci`-এর জন্য Lockfile, এবং `npm audit` বা অন্য Dependency Scan। (৩) Image Scan (যেমন Trivy) ও SBOM তৈরি; গুরুতর দুর্বলতায় ব্যর্থ। (৪) Image Registry-তে Push শুধু `main` Branch থেকে, শুধু সেই Job-এ `packages: write` অনুমতি। (৫) Cloud-এ Deploy-এর জন্য দীর্ঘমেয়াদি Key-এর বদলে OIDC-ভিত্তিক স্বল্পমেয়াদি Credential। (৬) Pull Request-এ Fork থেকে আসা Code-কে Secret দেখতে না দেওয়া। (৭) Branch Protection: CI পাস ও Review ছাড়া Merge নয়।

২৮.২০ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Readiness ব্যর্থ হলে সাধারণত কী ঘটে?

ক) Container Restart হয়

খ) Load Balancer এই Instance-এ নতুন Traffic পাঠানো বন্ধ করে

গ) Image মুছে যায়

ঘ) Database Restart হয়

প্রশ্ন ২. Dockerfile-এ `CMD ["node", "main.mjs"]` কেন `CMD npm start`-এর চেয়ে ভালো?

ক) দ্রুত Build হয়

খ) Image ছোট হয়

গ) `node` PID 1 হয়ে SIGTERM সরাসরি পায়

ঘ) npm Container-এ থাকে না

প্রশ্ন ৩. DeployReady-এ `PORT=abc` দিলে কী হয়?

ক) Port 3000-এ চালু হয়

খ) এলোমেলো Port-এ চালু হয়

গ) একটি Warning দিয়ে চালু হয়

ঘ) ভুলের তালিকা দেখিয়ে Exit Code 1

প্রশ্ন ৪. Production-এ Secret কোথায় রাখা উচিত?

ক) Platform-এর Secret Store-এ, নিয়ন্ত্রিত অনুমতি ও Rotation-সহ

খ) Git-এ, Private Repository হলে

গ) Docker Image-এর ENV-এ

ঘ) Frontend-এর Configuration File-এ

প্রশ্ন ৫. Compose-এ Drain ৫ সেকেন্ড ও Shutdown Timeout ১৫ সেকেন্ড হলে `stop_grace_period` কেমন হওয়া উচিত?

ক) ৫ সেকেন্ড

খ) ২০ সেকেন্ডের বেশি

গ) ১০ সেকেন্ড

ঘ) শূন্য

প্রশ্ন ৬. Metrics-এর Label-এ আসল Path (যেমন `/jobs/9381`) রাখলে কী সমস্যা?

ক) Metrics ভুল হয়

খ) Prometheus পড়তে পারে না

গ) অসংখ্য Time Series তৈরি হয়ে Memory শেষ হয় (High Cardinality)

ঘ) কোনো সমস্যা নেই

প্রশ্ন ৭. Canary Rollout-এ কী করা হয়?

ক) নতুন সংস্করণে প্রথমে অল্প Traffic, মেপে ধাপে বাড়ানো

খ) সব Traffic একসঙ্গে নতুন সংস্করণে

গ) Test বাদ দিয়ে দ্রুত Deploy

ঘ) পুরোনো সংস্করণ মুছে ফেলা

প্রশ্ন ৮. RPO কী বোঝায়?

ক) সেবা ফেরার সর্বোচ্চ সময়

খ) সর্বোচ্চ সহনীয় তথ্যহানি

গ) Request-এর গড় সময়

ঘ) Backup-এর আকার

প্রশ্ন ৯. SLO ৯৯.৯%, আর শেষ এক ঘণ্টায় Error Rate ১.৫%। Burn Rate কত?

ক) ১.৫

খ) ০.১

গ) ৯৯.৯

ঘ) ১৫

প্রশ্ন ১০. অপ্রত্যাশিত Error-এ DeployReady Client-কে কী পাঠায়?

ক) পুরো Stack Trace

খ) Database-এর Error বার্তা

গ) সাধারণ বার্তা ও Request ID; বিস্তারিত শুধু Log-এ

ঘ) কিছুই না, সংযোগ কেটে দেয়

২৮.২১ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ঘ · ৪–ক · ৫–খ · ৬–গ · ৭–ক · ৮–খ · ৯–ঘ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Restart Liveness-এর কাজ; Readiness শুধু Traffic সরায়।
২	গ	npm Signal ঠিকমতো পৌঁছায় না, ফলে Graceful Shutdown ঘটে না।
৩	ঘ	`loadConfig` সব ভুল জমিয়ে Throw করে; `main.mjs` Exit Code 1 দেয়।
৪	ক	Git ও Image-এর ইতিহাস মোছা কঠিন; Secret Store নিয়ন্ত্রিত ও Rotation-যোগ্য।
৫	খ	৫ + ১৫ = ২০; Grace Period কম হলে Platform আগেই SIGKILL পাঠায়।
৬	গ	প্রতিটি নতুন Label-মান একটি নতুন Time Series; তাই Route-এর নাম।
৭	ক	উদাহরণ ০৫-এর মতো সংখ্যা দেখে এগোনো বা ফিরে যাওয়া।
৮	খ	সময়ের সীমা RTO; RPO হলো কতটা তথ্য হারানো সহনীয়।
৯	ঘ	১.৫% ÷ ০.১% = ১৫; উদাহরণ ০৪-এ এই হারে Page।
১০	গ	`app.test.mjs`-এর Test দেখায় Response-এ "exploded" নেই, Log-এ আছে।

২৮.২২ অধ্যায়ের সারসংক্ষেপ

- Production মানে পুনরাবৃত্তিযোগ্য, যাচাইযোগ্য ও ফেরানো-যোগ্য Release, শুধু চালু Server নয়।
- Configuration Environment থেকে, শুরুতেই পুরো যাচাই; ভুল হলে চালুই নয়; গোপন মান কখনো Log-এ নয়।

- একবার Build, একই Image সবখানে; Commit SHA ও Digest দিয়ে পরিচয়; Lockfile ও `npm ci`।
- Dockerfile: নির্দিষ্ট Base, Cache-বান্ধব ক্রম, `.dockerignore`, Non-root, HEALTHCHECK, Exec Form CMD।
- Compose: Local-এ শুধু `127.0.0.1`, Secret `env_file`-এ, Drain + Timeout < Grace Period।
- CI: ন্যূনতম অনুমতি, Timeout, Test-এর পরে Image, Smoke Test; Action SHA দিয়ে Pin।
- Rolling, Blue/Green, Canary; Canary-র সিদ্ধান্ত সংখ্যায়; Migration পশ্চাৎ-সামঞ্জস্যপূর্ণ (Expand/Contract)।
- Liveness সরল, Readiness সত্যিকারের প্রস্তুতি; Graceful Shutdown-এর পাঁচ ধাপ।
- Structured Log ও Request ID; Prometheus Counter, Gauge, Histogram; সীমিত Label; Trace ও OpenTelemetry।
- SLI, SLO, Error Budget, Burn Rate; কাজে লাগে এমন Alert ও Runbook।
- Secret Store, Supply Chain, Least Privilege; RPO, RTO, Restore Drill, দোষারোপহীন Postmortem।

পরের অধ্যায়ে দৃষ্টি একটি Service থেকে পুরো ব্যবস্থায় যাবে: কখন Monolith ভাঙবেন, Service-গুলো কীভাবে কথা বলবে, একাধিক Service জুড়ে একটি Booking কীভাবে সম্পূর্ণ বা সম্পূর্ণ বাতিল হবে (Saga), এবং ব্যর্থতার মধ্যেও ব্যবস্থা কীভাবে নির্ভরযোগ্য থাকবে।

২৮.২৩ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Twelve-Factor App	Cloud-উপযোগী Application-এর বারোটি নীতি।
Fail Fast	ভুল Configuration-এ শুরুতেই থেমে যাওয়া।
Build Artifact	পরীক্ষিত, অপরিবর্তনীয় Release-বস্তু (যেমন Image)।
Image / Container	অপরিবর্তনীয় Template / তার চলমান Instance।
Image Digest	Image-এর বিষয়বস্তুর অনন্য Hash।
CI / CD	প্রতিটি পরিবর্তনের স্বয়ংক্রিয় যাচাই / স্বয়ংক্রিয় Release-প্রক্রিয়া।
Rolling, Blue/Green, Canary	Release ছড়ানোর তিনটি কৌশল।
Liveness / Readiness	Process জীবিত কি না / Traffic নিতে প্রস্তুত কি না।
Graceful Shutdown	চলমান কাজ শেষ করে সুশৃঙ্খলভাবে বন্ধ হওয়া।
Structured Log	প্রতি Line একটি যন্ত্র-পাঠযোগ্য Object।
Counter, Gauge, Histogram	শুধু বাড়ে / বাড়ে-কমে / Bucket-এ গোনা Metrics।
Trace, Span	একটি অনুরোধের পুরো পথ / তার একটি ধাপ।
SLI, SLO, SLA	মাপা সংখ্যা / লক্ষ্য / চুক্তি।

পরিভাষা	অর্থ
Error Budget, Burn Rate	অনুমোদিত ব্যর্থতা / তা খরচের গতি।
RPO, RTO	সহনীয় তথ্যহানি / সেবা ফেরার সময়সীমা।
SBOM	Software-এ থাকা সব Component-এর তালিকা।

২৮.২৪ আরও পড়ুন

- **The Twelve-Factor App:** `https://12factor.net/` — মূল নীতিমালা।
- **Docker — Building best practices:** `https://docs.docker.com/build/building/best-practices/` — Dockerfile লেখার পরামর্শ।
- **Docker — Compose file reference:** `https://docs.docker.com/reference/compose-file/` — Compose-এর প্রতিটি Key।
- **GitHub — Security hardening for GitHub Actions:** `https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions` — অনুমতি, Pin ও Secret।
- **Prometheus — Exposition formats:** `https://prometheus.io/docs/instrumenting/exposition_formats/` — Text Format।
- **OpenTelemetry — Concepts:** `https://opentelemetry.io/docs/concepts/` — Trace, Metrics, Log।
- **Google SRE Workbook — Alerting on SLOs:** `https://sre.google/workbook/alerting-on-slos/` — Burn Rate-ভিত্তিক Alert।
- **Node.js — Process signal events:** `https://nodejs.org/api/process.html#signal-events` — SIGTERM ও SIGINT।